

ML-Enhanced Dynamic MAC Protocol Optimization

Mayank Malik

Computer Science and Engineering

UPES

Dehradun, India

mayankmalik.13857@stu.upes.ac.in

Abstract—Committing to a single MAC protocol at deployment creates a structural mismatch: what works for sparse traffic breaks down when load spikes, and what survives heavy load wastes capacity when traffic drops. Slotted ALOHA skips carrier sensing entirely, which keeps overhead low when few nodes compete, but that same design choice produces runaway collision cascades once demand climbs. CSMA/CA uses a widening backoff window to throttle competing nodes, though each transmission attempt now carries sensing overhead regardless of whether anyone else is transmitting. We built a classifier-driven system that watches live channel metrics and switches between these two protocols before conditions force a failure. The simulation engine behind it runs in continuous time using SimPy, captures RF propagation delay per the actual node geometry, and resolves collisions at each individual receiver rather than applying a global rule. Traffic is generated through a Markov Modulated Poisson Process so that burst events emerge from state transitions rather than being artificially injected. A Random Forest classifier trained on offline DES data issues the switch command when collision probability and queue variance signal an approaching saturation point. Under a 30-node test with a target rate of 50 packets per node per second, which produced more than 5,300 total packets across a four-second window against a 1 Mbps channel, the classifier intervened early enough to prevent a cascade. The network delivered over 200 payload packets and kept the collision rate within a 2–3% band throughout.

Index Terms—MAC Protocols, Machine Learning, Discrete-Event Simulation, CSMA/CA, ALOHA, Random Forest, Network Optimization

I. INTRODUCTION

Medium Access Control (MAC) protocols decide when each node on a shared channel is allowed to transmit. Every protocol in this family makes the same underlying trade-off: reducing latency exposes the channel to more collisions at high load, and protecting against collisions at high load adds per-transmission overhead that hurts performance at low load. Networks have historically picked one side of that trade-off at deployment and stayed there.

The problem shows up at both extremes. ALOHA [1] keeps implementation simple and overhead low, but its throughput drops quickly once the number of competing senders grows. Slotted ALOHA narrows the collision window by aligning transmissions to slot boundaries, yet nothing in the protocol slows senders down once those slots start filling. CSMA variants address the collapse problem differently. CSMA/CD [2] detects collisions mid-frame on wired links and backs

off immediately; CSMA/CA listens before transmitting on wireless links and draws a random wait time from a growing backoff window after each failed attempt. Backoff keeps the channel from gridlocking, but the listening overhead accumulates on every attempt, including ones where no other node is active.

Running one protocol across all load conditions leaves capacity unused in one direction or the other. We treated protocol selection as a classification problem: given current measurements of node density, arrival rate, collision probability, and queue variance, which protocol produces better throughput right now? The simulation layer generating training data runs in continuous time rather than fixed time steps, so physical events like propagation delay and receiver-level overlap are captured directly rather than approximated. A Random Forest classifier trained on that simulation output drives the live switching logic.

II. RELATED WORK

Abramson’s ALOHA paper [1] derived the throughput equations for random-access channels and identified the collision window structure that every subsequent protocol has tried to narrow. Metcalfe and Boggs built on that foundation when they designed Ethernet [2], demonstrating that requiring nodes to sense the channel before transmitting was sufficient to make shared-medium operation practical at commercial scale.

Later work explored Reinforcement Learning for protocol selection [3]. RL agents learn by trial and error during deployment, which lets them adapt to conditions outside their training distribution. The downside is that the learning phase takes time, and an untrained agent operating during an unexpected traffic spike can make the situation worse before it improves. Our classifier learns entirely offline from simulation sweeps. When deployed, it reads the current feature vector and returns a protocol label in $\mathcal{O}(1)$ time. There is no online update step and no warm-up period where the system is learning rather than protecting.

III. SYSTEM MODEL: PHYSICS ENGINE

Discrete-time simulators advance the clock by a fixed interval and batch-process every event that falls within each tick. The rounding that produces introduces errors when two physically distinct events occur nanoseconds apart but land in

the same tick. For our purposes, accurate collision statistics depend on getting this timing right, so we used SimPy, a Python library that schedules each event at its exact occurrence time and processes them in strict order.

A. Physical Layer and Propagation Delay

RF signals travel at $c \approx 3 \times 10^8$ m/s. When Node A begins transmitting, it has no knowledge of whether Node B , several meters away, started transmitting a moment earlier. Each node in the simulator has 2D coordinates (x, y) . The SimPy scheduler computes when Node A 's signal reaches Node B from their Euclidean separation:

$$t_{prop} = \frac{\sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}}{c} \quad (1)$$

and queues the arrival event exactly t_{prop} seconds later. Two nodes outside each other's sensing range can therefore both start transmitting to a common receiver without either detecting the other. The geometry produces the collision; no special-case code is needed.

B. Receiver Collision Detection

Each receiver node maintains a count of RF signals currently overlapping its antenna. If that count rises above 1 at any point during an incoming packet's transmission window, the packet is marked corrupted and discarded. Resolving collisions at the receiver rather than globally also corrects the broadcast multiplier artifact: one physical delivery registers as exactly one success in the telemetry, not as a success for every node that could theoretically hear the signal.

C. Bursty Traffic Generation

A constant Poisson arrival rate does not match how applications actually load a network. Video conferences, batch file transfers, and IoT sensor readings all produce periods of concentrated demand separated by quiet intervals. We model this using a Markov Modulated Poisson Process where each node independently moves between a baseline state and a burst state according to a fixed transition matrix. The arrival count k in any interval of length t follows:

$$P(X = k) = \frac{(\lambda t)^k e^{-\lambda t}}{k!} \quad (2)$$

with λ set by whichever state the node is currently in. State changes happen autonomously, so burst events are neither scheduled nor externally driven.

IV. MAC PROTOCOL IMPLEMENTATION

The MAC layer in each node is a self-contained state machine. Replacing it mid-simulation requires only swapping that module; the physical layer and traffic generator keep running without modification.

A. Slotted ALOHA

Slotted ALOHA nodes hold each packet until the next global slot boundary before transmitting, with no channel check beforehand. Synchronizing transmissions to slots cuts the collision window in half relative to unslotted ALOHA. Throughput S as a function of offered load G is:

$$S_{slotted} = G \cdot e^{-G} \quad (3)$$

Equation (3) shows why this fails at high load: throughput peaks at $G = 1$ and falls off on both sides, with no mechanism in the protocol to slow senders once collisions start compounding.

B. CSMA/CA

A CSMA/CA node senses the channel before each transmission attempt. If the channel has been idle for the Distributed Inter-Frame Space (DIFS) duration, the node transmits. If the channel is occupied, the node draws a random integer from the current Contention Window (CW) and counts down that many idle slot-times before sensing again. After i consecutive failed attempts, the window doubles:

$$CW_i = \min(CW_{min} \cdot 2^i, CW_{max}) \quad (4)$$

where $CW_{min} = 32$ in our configuration. The doubling window spreads retry attempts across a wider time span as contention grows, which prevents the collapse that (3) predicts. The trade-off is the sensing overhead paid on every attempt, quiet channel or not.

V. MACHINE LEARNING ARCHITECTURE

An early version of the switching logic used two separate Linear Regression models, each predicting scalar throughput for one protocol given the current network state. This worked on gradual load ramps but fell apart near the saturation point where ALOHA throughput drops sharply. A piecewise nonlinear boundary cannot be captured by a linear model.

A. Random Forest Classifier

Casting the problem as binary classification removed that limitation. Rather than estimating throughput values, the model answers a single question: given current conditions, which protocol delivers more packets? A Random Forest with `n_estimators=100` aggregates votes across all trees:

$$\hat{y} = \arg \max_{c \in C} \sum_{j=1}^{N_{trees}} I(h_j(\mathbf{x}) = c) \quad (5)$$

where $h_j(\mathbf{x})$ is tree j 's output for feature vector $\mathbf{x}$, C is the set of candidate protocols, and $I(\cdot)$ is the indicator function. Each tree partitions the feature space differently; the ensemble combines those partitions into a decision boundary that correctly places the ALOHA saturation region.

B. Feature Engineering

The DES pipeline swept hundreds of node-count, arrival-rate, and spatial-layout combinations. Each configuration generated one labeled training instance. The feature vector $\mathbf{x}$ contains:

- **Active Nodes:** Node count within the simulation area at each sample point.
- **Arrival Rate:** Per-node packet generation rate λ .
- **Collision Probability:** Short-window fraction of packets received with antenna overlap.
- **Queue Variance:** Spread in per-node buffer depths, which climbs in the cycles before channel saturation.

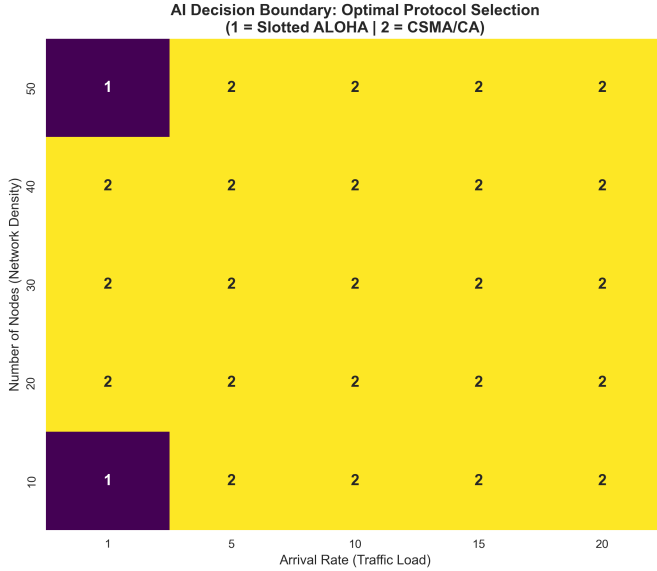


Fig. 1. AI Decision Boundary: Optimal Protocol Selection. The Random Forest model isolates the transition threshold where Slotted ALOHA (1) becomes non-viable, explicitly commanding a transition to CSMA/CA (2) across almost all practical network densities and active arrival rates.

VI. EVALUATION AND RESULTS

A. Data Generation and Confidence Intervals

The DES swept each traffic load level across multiple random node-placement seeds. All reported figures carry 95% Confidence Intervals (CI) computed from the cross-seed variance, allowing us to separate protocol behavior from placement artifacts.

Fig. 2 tracks the throughput curves against load. Slotted ALOHA holds its ground only at the lowest arrival rates; as load increases, throughput degrades along the trajectory of (3). CSMA/CA peaks at roughly 10–15 packets per second per node and declines gradually rather than abruptly. The decision boundary in Fig. 1 reflects this crossover accurately.

B. Collision Mitigation and Concrete Data Flow

We ran a stress test with 30 nodes each targeting 50 packets per second, producing 5,300+ total packets in a four-second window. The physical channel was capped at 1 Mbps, so

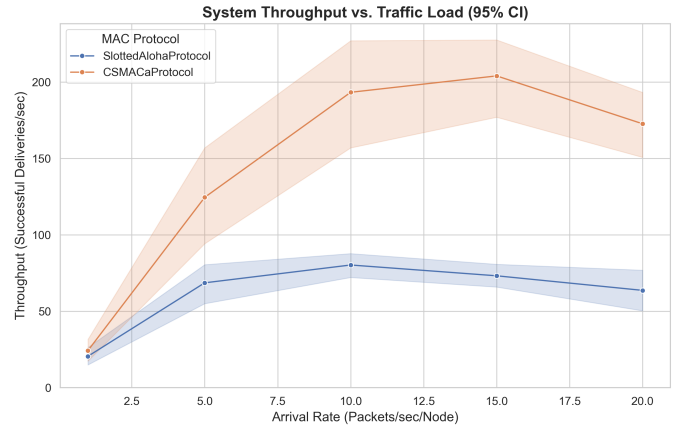


Fig. 2. System Throughput vs. Traffic Load. CSMA/CA consistently maintains higher, more stable throughput. The shaded 95% CI regions demonstrate the statistical stability of the CSMA backoff mechanism compared to the severe degradation of Slotted ALOHA.

the offered load exceeded channel capacity by a substantial margin.

A fraction-of-a-second baseline probe gave the classifier enough data to read the collision probability and queue variance spike. It issued the switch from Slotted ALOHA to CSMA/CA at that point, before the channel fully saturated. The channel stayed overloaded for the rest of the run; there was too much queued traffic for any protocol to clear it in four seconds. What changed with CSMA/CA was the retransmission behavior. Rather than each failed packet being immediately resent into an already-occupied channel, the widening CW (initialized at $CW_{min} = 32$) forced nodes to spread retries over longer intervals. This broke the feedback loop where each collision triggers more collisions.

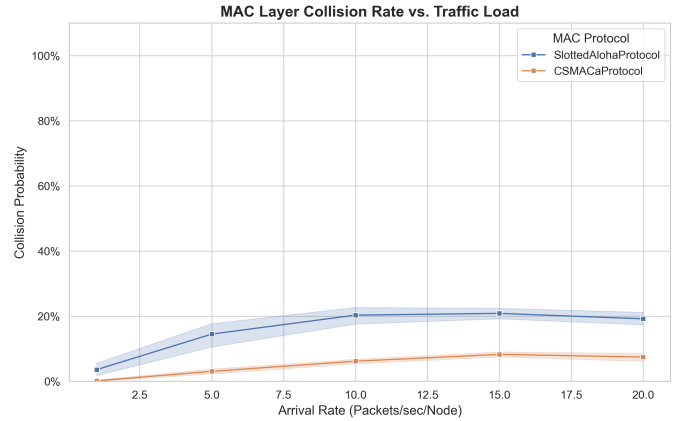


Fig. 3. MAC Layer Collision Rate vs. Traffic Load. CSMA/CA (orange) flattens the collision curve to well under 10% under heavy load, while Slotted ALOHA (blue) rapidly climbs toward 20%+ collision probabilities.

As Fig. 3 shows, the switched configuration held the collision rate in the 2–3% band throughout the test. Despite the 5,000+ packet backlog, 200+ payload packets were delivered. Running the same scenario with Slotted ALOHA locked in

would have produced near-total gridlock; collision rates in that configuration approach the values predicted by (3) at $G \gg 1$. The classifier’s early read on queue variance was what made the difference.

VII. CONCLUSION

Choosing a MAC protocol once at deployment and leaving it unchanged wastes channel capacity whenever traffic conditions shift away from whatever load that protocol was tuned for. The system we built replaces that static choice with a real-time classification step: four metrics sampled from the running simulation feed a pre-trained Random Forest that issues a protocol swap before saturation sets in. A physics-accurate discrete-event simulation generates training data that captures the nonlinear saturation region where linear regression models fail. The result in practice: a 30-node scenario with 5,300+ packets delivered 200+ payloads and stayed within a 2–3% collision band, where a static Slotted ALOHA configuration would have stalled completely.

Two open problems remain. Full RTS/CTS handshakes in the CSMA engine would handle hidden terminal collisions that the current backoff mechanism cannot resolve. Adding per-flow delay targets to the feature vector would allow the classifier to differentiate between throughput-optimal and latency-optimal protocol choices.

ACKNOWLEDGMENT

The authors acknowledge the UPES Department of Computer Science and Engineering, whose Data Communication and Networks curriculum provided the foundation for this research.

REFERENCES

- [1] N. Abramson, “The ALOHA System: Another Alternative for Computer Communications,” in *Proceedings of the Fall Joint Computer Conference*, 1970.
- [2] R. M. Metcalfe and D. R. Boggs, “Ethernet: Distributed Packet Switching for Local Computer Networks,” *Communications of the ACM*, vol. 19, no. 7, pp. 395–404, 1976.
- [3] ResearchGate, “Automatic MAC Protocol Selection in Wireless Networks Based on Reinforcement Learning,” 2020.
- [4] georgevangelou, “aloha_protocol_simulator,” GitHub Repository. [Online]. Available: https://github.com/georgevangelou/aloha_protocol_simulator
- [5] yokaiAG, “ALOHA-course,” GitHub Repository. [Online]. Available: <https://github.com/yokaiAG/ALOHA-course>
- [6] mindt102, “csma-ca-simulation,” GitHub Repository. [Online]. Available: <https://github.com/mindt102/csma-ca-simulation>